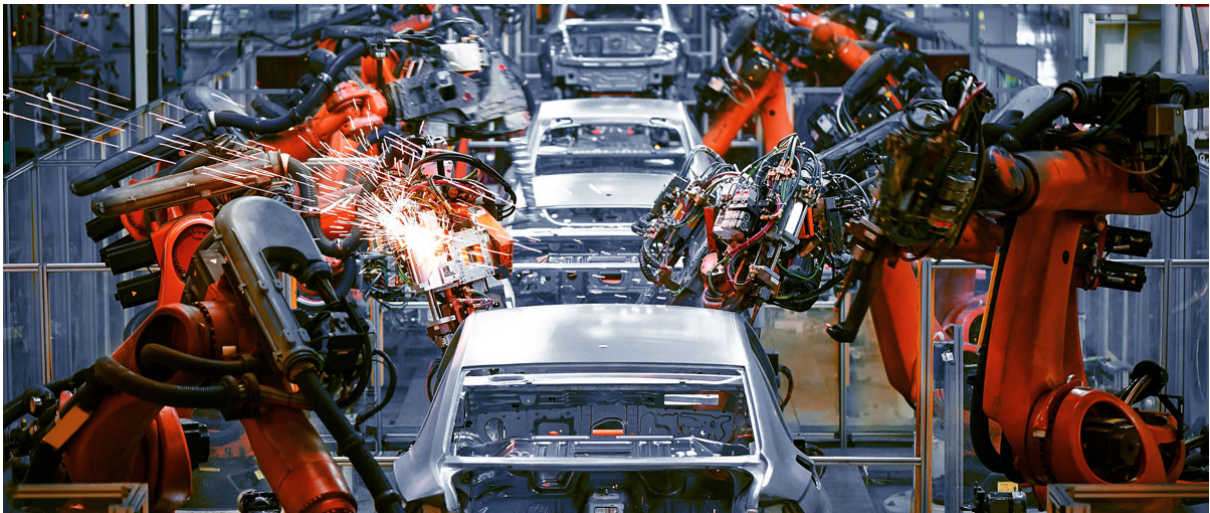


Chapter 6: Automating Feature-Level Exploration

<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to design and implement reusable functions that iterate through DataFrame columns and compute appropriate statistics based on data type
<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to use branching logic to differentiate between numeric and categorical processing paths within automated exploration functions
<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to write dynamic, error-resistant code that adapts to datasets of varying sizes and column compositions without manual modification
<{<http://www.bookeducator.com/Textbook>}learningobjective >Students will be able to package automation functions in external Python modules for reuse across multiple projects

6.1 Introduction



You have already learned useful univariate statistics and visualizations that need to be calculated during the Data Understanding (a.k.a. EDA) phase. Writing them in Python (instead of using a calculator or doing them by hand) is an important first step—but you can go further by automating the entire workflow. The goal of this chapter is to help you build reusable code that quickly summarizes any dataset, so you spend less time rewriting the same analysis and more time interpreting results.

Of course, every dataset has unique quirks that you can't fully automate. But many tasks repeat across projects, especially when they depend on the data type of each column. For example, earlier you checked whether each column in the insurance dataset was numeric:

```
print('age: ' + str(pd.api.types.is_numeric_dtype(df.age)))
print('sex: ' + str(pd.api.types.is_numeric_dtype(df.sex)))
print('bmi: ' + str(pd.api.types.is_numeric_dtype(df.bmi)))
print('children: ' + str(pd.api.types.is_numeric_dtype(df.children)))
print('smoker: ' + str(pd.api.types.is_numeric_dtype(df.smoker)))
print('region: ' + str(pd.api.types.is_numeric_dtype(df.region)))
print('charges: ' + str(pd.api.types.is_numeric_dtype(df.charges)))
```

That approach works, but it doesn't scale. The repeated pattern is the clue: when you see yourself copying and pasting the same code with only a small change (like a different column name), it's time to automate. One simple improvement is to loop through the columns:



```
# Print them using a loop
for col in df.columns:
    print(str(col) + ': ' + str(pd.api.types.is_numeric_dtype(df[col])))

# Output:
# age: True
# sex: False
# bmi: True
# children: True
# smoker: False
# region: False
# charges: True
```

Loops reduce repetition, but they're only the beginning. In this chapter, you'll take the next step by wrapping these ideas into functions that are reusable, dynamic across datasets, and less likely to break when the data changes.

Images in this section were created using DALL·E from OpenAI.

6.2 What Is Automation?

This video can be viewed [online](#).

Automation in data mining is a cornerstone of effective data science. By automating processes, we can save time, minimize repetitive tasks, and reduce the likelihood of human error. As a data scientist, always seek opportunities to make your code:

To keep this chapter grounded, we will use one running example: imagine you receive a new dataset every week with 200+ columns. Manually checking data types, missing values, and distributions is slow and error-prone.

Automation helps you write that logic once and reuse it across every dataset.

1. **Automated**
2. **Dynamic**
3. **Error-resistant**

These three ideas are related, but not identical: **automation** reduces repetition, **dynamic code** adapts to new datasets without manual rewrites, and **error-resistant code** keeps working even when inputs are messy or incomplete.

For simplicity, I may use the term *automation* or *automated* to encompass one or all three principles: automation itself, dynamic processing, and error resistance. While these concepts differ, they work together to create efficient, scalable, and reliable code.

What Does Automation Mean?

automation The process of replicating human effort and decision-making in programming code. refers to reducing human effort by turning repeated steps into code that runs the same way every time. In practice, automation usually combines iteration (loops), decision logic (if/else), and consistent outputs. For example, instead of manually typing code for each column in a dataset to determine its data type, you can write a loop that checks all columns automatically. This approach eliminates redundant code and minimizes manual intervention.

In essence, automation replaces manual work with systematic, repeatable processes. Some automation simply saves time, while other automation includes decision-making logic (for example, choosing a different analysis path for numeric versus categorical data). That kind of automated decision-making becomes especially important later when you begin building predictive models.

What Is Dynamic Code?

Code that is **dynamic** In software, code that will work uninterrupted regardless of the amount or type of inputs that are provided. adapts to varying inputs and conditions without requiring manual adjustments. Dynamic processes ensure that your code works even when the dataset changes in:

1. **Input Data:** Whether the input dataset has different values or formats.
2. **Dataset Structure:** The number of columns, rows, or other structural features.
3. **Data Types:** Whether columns contain numeric, categorical, or mixed data.

For instance, a dynamic function designed to analyze datasets should handle small datasets as effectively as large ones, regardless of how many columns exist or what types they contain. In our weekly “200-column dataset” example, dynamic code means you do not have to rewrite your analysis when a new column is added, removed, renamed, or stored with a different data type.

Dynamic coding is essential for scalable solutions because it makes your work reusable across projects and resilient to change.

What Does Error-Resistant Mean?

Finally, `error-resistant` In the context of software, this refers to code that will work uninterrupted even if the user attempts to submit invalid inputs either by (1) specifying a better form for the inputs or (2) adapting or modifying the inputs to an acceptable form. code anticipates and handles issues that may arise during execution. In real datasets, common problems include missing columns, unexpected data types, empty datasets, and columns with all missing values. You can make your code error-resistant by:

1. **Validating Inputs:** Ensuring that the data meets required conditions before processing begins (for example, confirming an input is a DataFrame and contains expected columns).
2. **Adapting Inputs:** Transforming inputs into acceptable forms when possible (for example, handling missing values by imputing them, skipping invalid columns, or excluding affected rows).

Error resistance minimizes crashes and unexpected behavior, making your code more reliable and user-friendly. In our weekly dataset example, error-resistant code means your analysis still runs even if the dataset arrives with a

missing column, a misspelled label, or a column that is unexpectedly stored as text instead of numbers.

Why Automation Matters

Automation is more than just a time-saver—it's a fundamental aspect of modern data science. Effective automation allows you to:

- Eliminate repetitive tasks.
- Handle datasets of varying sizes and structures.
- Create reusable, parameterized scripts.
- Reduce errors and ensure robust processing under diverse conditions.

As a data scientist, mastering automation will enable you to tackle complex problems more efficiently and develop scalable solutions that adapt to real-world challenges.

In the next section, you'll take these concepts and implement them by building Python functions that automatically analyze every column in a dataset.

6.3 Automating Univariate Stats

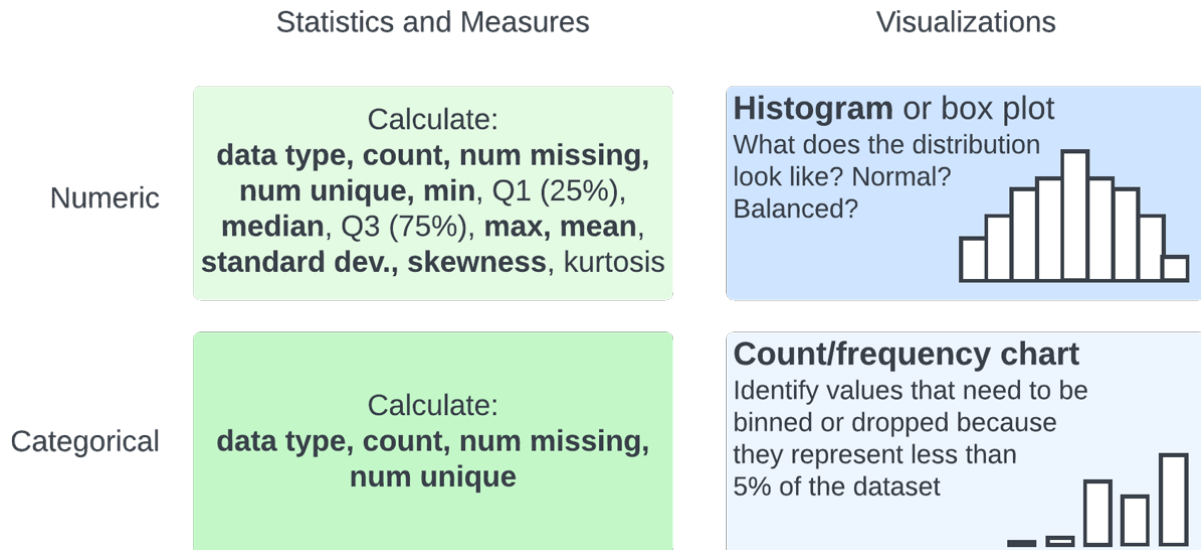


Figure 6.1: Summary of Univariate Statistics and Visualizations to Automate

The image above summarizes the list of statistics, measures, and visualizations that we typically want to create (the minimum in bold) as we explore a dataset. But where should we begin this process of automating stats and charts?

In the prior section, you learned that strong >automated (reduces repetition), **dynamic** (works across datasets), and **error-resistant** (fails gracefully when the data is messy). In this section, you'll apply those ideas by building a function that generates univariate statistics for **every** column in a DataFrame.

We'll start small, test early, and improve the function step-by-step. This is what real automation looks like: build a reliable base, then extend it with careful branching, clean outputs, and reusable packaging.

A Repeatable Automation Pattern

Automating the process of generating univariate statistics improves efficiency and consistency. If you're unsure where to start, use this general

pattern. With practice, it will become second nature:

1. Define the automation function.
2. Import necessary Python packages.
3. Create variables for processing.
4. Define the iteration (e.g., a loop).
5. Perform processing for each iteration.
6. Define the decision criterion.
7. Perform processing based on the decision structure.
8. Synthesize and return the results.

Design Checklist (Before You Code)

Before writing the full function, decide what “success” looks like. At minimum: (1) input is a DataFrame, (2) output is a clean summary table, and (3) the function works even when columns are messy (missing values, unexpected dtypes, etc.).

A. Define the Function Interface

Step 1: Define the Automation Function

The first step is to define the function. This includes deciding on its input parameter(s) and output. Ask yourself:

1. What input data is necessary for the function to perform its task?
2. What should the function return?

In this case, we want a function that automates univariate statistics for all columns in a DataFrame. Thus, our input will be the entire DataFrame (**df**).

Here's an initial implementation:



```
# Step 1: Define the Automation Function
def unistats(df):
    return df
```

Although simple, it's crucial to choose input parameters wisely. When designing functions, balance two criteria:

1. Minimize the required input to what's strictly necessary.
 - For example, don't require a Pandas Series if a basic Python list would work.
2. Choose input types that minimize additional preprocessing.
 - For example, pass a Pandas Series if you need row labels or convenient methods like `.nunique()`.

Notice how these criteria can conflict. In this context, we want a function that can analyze **all** features automatically, so passing the full DataFrame (**df**) is the best choice.

Step 2: Import Python Packages

A function should be self-contained, meaning it cannot assume that the required packages are already available in memory. Therefore, explicitly import all packages needed for the function.

For our `unistats` function, we'll use Pandas to manage the DataFrame and calculate statistics:



```
def unistats(df):  
    # Step 2: Import necessary packages  
    import pandas as pd  
  
    return df
```

Step 3: Create Variables for Processing

Next, consider the structure of the output. Will the function return a simple value (e.g., int, float, str, bool) or a collection (e.g., list, dict, DataFrame)? If it's a collection, initialize it before the loop begins.

For unistats, the output is a summary table containing univariate statistics for each column. We initialize it as an empty DataFrame:



```
def unistats(df):  
    import pandas as pd  
  
    # Step 3: Create a variable for processing  
    output_df = pd.DataFrame()  
  
    return output_df
```

B. Iterate and Compute Baseline Stats

Step 4: Define the Iteration

Data science tasks often involve applying repeated processes to columns or rows. For unistats, we need to iterate through each column of the DataFrame:



```
def unistats(df):  
    import pandas as pd
```

```

output_df = pd.DataFrame(columns=['Type'])

# Step 4: Define the iteration
for col in df.columns:
    pass # Temporary placeholder

return output_df

```

This loop enables us to process each column individually.

Step 5: Perform Processing for Every Iteration

Some operations apply to all columns, regardless of whether they contain numeric or categorical data. For instance:

- Counting non-missing values.
- Counting unique values.
- Identifying the data type.

These operations form the base of our `unistats` function. Update it as follows:



```

def unistats(df):
    import pandas as pd

    output_df = pd.DataFrame(columns=['Count', 'Unique', 'Type'])

    for col in df.columns:
        count = df[col].count()      # non-missing count
        unique = df[col].nunique()   # unique values
        dtype = str(df[col].dtype)   # dtype as a readable string

        output_df.loc[col] = [count, unique, dtype]

    return output_df

```

Now it's time to test. Don't wait until you think you've finished to test your code—test early and often. Below is a quick test call:



```
# Test out the function:
import pandas as pd
df = pd.read_csv('http://www.ishelp.info/data/insurance.csv')
unistats(df)
```



Each time through the loop, a new row is added to `output_df`. You can think of it as “rotating” the DataFrame: columns in `df` become row labels in `output_df`. This is a common and readable summary format.

If you prefer the original orientation (columns stay as columns), you can build lists inside the loop and create the DataFrame after the loop finishes. Here’s one way to do it (optional):



```
def unistats(df):
    import pandas as pd

    counts = []
    uniques = []
    dtypes = []

    for col in df.columns:
        counts.append(df[col].count())
        uniques.append(df[col].nunique())
        dtypes.append(str(df[col].dtype))

    output_df = pd.DataFrame(
        [counts, uniques, dtypes],
        columns=df.columns,
        index=['Count', 'Unique', 'Type']
    )

    return output_df
```



```
# Test out the function:
import pandas as pd
df = pd.read_csv('http://www.ishelp.info/data/insurance.csv')
unistats(df)
```



Which do you prefer? Use either format for the rest of this tutorial. For consistency, the rest of the examples will use the first format (columns become rows).

C. Branch by Data Type

Step 6: Define the Decision Criterion

To perform operations specific to numeric or categorical data, we need a decision criterion. Pandas provides `pd.api.types.is_numeric_dtype()` to identify numeric columns. A practical and consistent pattern is to evaluate the Series itself:



```
def unistats(df):
    import pandas as pd

    output_df = pd.DataFrame(columns=['Count', 'Unique', 'Type'])

    for col in df.columns:
        count = df[col].count()
        unique = df[col].nunique()
        dtype = str(df[col].dtype)

        # Decision criterion: is the column numeric?
        if pd.api.types.is_numeric_dtype(df[col]):
            print(""Testing: " + col + " is numeric"")

        output_df.loc[col] = [count, unique, dtype]

    return output_df
```

Let's test it out:



```

unistats(df)

# Output:
# Testing: age is numeric
# Testing: bmi is numeric
# Testing: children is numeric
# Testing: charges is numeric

```

Notice that the print statements execute from inside the function, and the DataFrame is still returned when the function finishes.

D. Add Branch-Specific Processing and Fix a Common Logical Bug

Step 7: Perform Processing in Each Branch

Now that we can control the flow based on data type, we can add additional processing for numeric columns. The key idea is: numeric columns get numeric statistics; categorical columns do not.

In the next section, you'll extend this idea with more advanced automation patterns. For now, we'll keep the function focused: baseline stats for every column, plus common numeric stats for numeric columns.



```

def unistats(df):
    import pandas as pd

    output_df = pd.DataFrame(columns=[
        'Count', 'Unique', 'Type',
        'Min', 'Max', '25%', '50%', '75%',
        'Mean', 'Median', 'Mode', 'Std', 'Skew', 'Kurt'
    ])

    for col in df.columns:
        count = df[col].count()
        unique = df[col].nunique()
        dtype = str(df[col].dtype)

        # Initialize branch-specific values to placeholders each iteration
        min_val = '-'

```

```

max_val = '-'
q1 = '-'
q2 = '-'
q3 = '-'
mean_val = '-'
median_val = '-'
mode_val = '-'
std_val = '-'
skew_val = '-'
kurt_val = '-'

if pd.api.types.is_numeric_dtype(df[col]):
    min_val = round(df[col].min(), 2)
    max_val = round(df[col].max(), 2)
    q1 = round(df[col].quantile(0.25), 2)
    q2 = round(df[col].quantile(0.50), 2)
    q3 = round(df[col].quantile(0.75), 2)
    mean_val = round(df[col].mean(), 2)
    median_val = round(df[col].median(), 2)

    # Mode can return multiple values; we'll take the first as a simple default
    mode_series = df[col].mode()
    mode_val = round(mode_series.values[0], 2) if len(mode_series) > 0 else '-'

    std_val = round(df[col].std(), 2)
    skew_val = round(df[col].skew(), 2)
    kurt_val = round(df[col].kurt(), 2)

output_df.loc[col] = (
    count, unique, dtype,
    min_val, max_val, q1, q2, q3,
    mean_val, median_val, mode_val, std_val, skew_val, kurt_val
)

return output_df

import pandas as pd
df = pd.read_csv('http://www.ishelp.info/data/insurance.csv')
unistats(df)

```



If you wrote a version of this function that “looked right” but produced repeated numeric values for categorical columns, you just encountered one of the most common logical bugs in automation: **values from a prior loop iteration were reused**.

This bug happens when you only assign numeric-stat variables inside the numeric branch and never reset them in the categorical branch. The loop

keeps the old values in memory, so categorical columns accidentally inherit numeric values from the prior numeric column.

A reliable fix is exactly what you see above: set placeholders for every branch-specific variable at the start of each loop iteration, and then overwrite them only when the column is numeric.

If you'd like to compare approaches, here are two classic options for handling categorical columns (both are valid). The first “zeros out” variables in an else branch. The second writes placeholders directly to the output table.



```
# Option 1: Use else to reset non-applicable values
def unistats(df):
    import pandas as pd

    output_df = pd.DataFrame(columns=[
        'Count', 'Unique', 'Type',
        'Min', 'Max', '25%', '50%', '75%',
        'Mean', 'Median', 'Mode', 'Std', 'Skew', 'Kurt'
    ])

    for col in df.columns:
        count = df[col].count()
        unique = df[col].nunique()
        dtype = str(df[col].dtype)

        if pd.api.types.is_numeric_dtype(df[col]):
            min_val = round(df[col].min(), 2)
            max_val = round(df[col].max(), 2)
            q1 = round(df[col].quantile(0.25), 2)
            q2 = round(df[col].quantile(0.50), 2)
            q3 = round(df[col].quantile(0.75), 2)
            mean_val = round(df[col].mean(), 2)
            median_val = round(df[col].median(), 2)

            mode_series = df[col].mode()
            mode_val = round(mode_series.values[0], 2) if len(mode_series) > 0 else '-'

            std_val = round(df[col].std(), 2)
            skew_val = round(df[col].skew(), 2)
            kurt_val = round(df[col].kurt(), 2)
        else:
            min_val = '-'
            max_val = '-'
            q1 = '-'
            q2 = '-'
            q3 = '-'
            mean_val = '-'
            median_val = '-'
```

```

        mode_val = '-'
        std_val = '-'
        skew_val = '-'
        kurt_val = '-'

    output_df.loc[col] = (
        count, unique, dtype,
        min_val, max_val, q1, q2, q3,
        mean_val, median_val, mode_val, std_val, skew_val, kurt_val
    )

return output_df

```



```

# Option 2: Write placeholders directly to the output table
def unistats(df):
    import pandas as pd

    output_df = pd.DataFrame(columns=[
        'Count', 'Unique', 'Type',
        'Min', 'Max', '25%', '50%', '75%',
        'Mean', 'Median', 'Mode', 'Std', 'Skew', 'Kurt'
    ])

    for col in df.columns:
        count = df[col].count()
        unique = df[col].nunique()
        dtype = str(df[col].dtype)

        if pd.api.types.is_numeric_dtype(df[col]):
            min_val = round(df[col].min(), 2)
            max_val = round(df[col].max(), 2)
            q1 = round(df[col].quantile(0.25), 2)
            q2 = round(df[col].quantile(0.50), 2)
            q3 = round(df[col].quantile(0.75), 2)
            mean_val = round(df[col].mean(), 2)
            median_val = round(df[col].median(), 2)

            mode_series = df[col].mode()
            mode_val = round(mode_series.values[0], 2) if len(mode_series) > 0 else '-'

            std_val = round(df[col].std(), 2)
            skew_val = round(df[col].skew(), 2)
            kurt_val = round(df[col].kurt(), 2)

            output_df.loc[col] = (
                count, unique, dtype,
                min_val, max_val, q1, q2, q3,
                mean_val, median_val, mode_val, std_val, skew_val, kurt_val
            )
        else:
            output_df.loc[col] = (count, unique, dtype, '-', '-', '-', '-', '-', '-', '-',
            '-', '-', '-', '-')

    return output_df

```



E. Synthesize and Reuse the Function

Step 8: Synthesize and Return Results

In this workflow, synthesis happens as we add one completed row per column into `output_df`. The final step is returning the completed summary table:

```
return output_df
```

Where Should We Keep `unistats`?

To reuse the function, save it in an external script (for example, `my_functions.py`) and import it into your projects. In the example below, the file is saved in the “Colab Notebooks” folder of Google Drive:



```
import pandas as pd
import sys

sys.path.append('/content/drive/My Drive/Colab Notebooks/')
import my_functions as my # Do not include ".py" in the import

df = pd.read_csv('http://www.ishelp.info/data/insurance.csv')
my.unistats(df)
```



6.4Practice

Next, see how you do with these practice problems to assess your ability to automate:

Practice #1: Automate Univariate Visualization

Create a function to automate the generation of bar charts and histograms. This could be either a stand-alone function that generates all univariate charts for a given dataset or a "child" function that is called from the `unistats` function. Add the following capabilities:

- Allow the caller to choose either a box plot, histogram, or both charts to be generated for each feature.
- Allow the caller to choose whether or not relevant univariate statistics should be embedded to the chart.
- Allow the caller to choose whether or not (and where) the chart to be saved as a .png image file.

Use the same `insurance.csv` dataset you learned with in this chapter to test this function.



Practice #2: Automated Binning

Binning numeric data into grouped ranges (1, 2, 3, 4, 5, 6 -> [1 to 3], [4 to 6]) is a common task in data science. It is necessary for creating visualizations such as histograms and bar charts. In addition, when numeric data is highly skewed, one solution is to recode the data into a theoretically-specified number of bins. But to do so, you must (1) identify the optimal number of bins and (2) create a new version of the list using the bin cutoff values to indicate which bin they belong to. For example, the list below can be binned into three groups as follows:

```
original_list = [1, 1, 1, 2, 2, 3, 32, 36, 39, 42, 42, 44, 68, 68, 68, 70, 89, 92]
binned_list = [31, 31, 31, 31, 31, 31, 61, 61, 61, 61, 61, 61, 92, 92, 92, 92, 92, 92]
```

The new list represents three equally sized groups. The numbers 31, 61, and 92 are the upper "edges" of the bins.

Your task is to write a function that will determine the number of bins required for a given list. Your task is to create a function that takes a list as an input parameter and returns a list of the bin edges. For example, if the first list above was inputted to your function, it would return [31, 61, 92], indicating that three bins were required, and the bin width was approximately 31. The formula used to determine the appropriate number of bins is the time-honored square root of n rule. That means the appropriate number of bins is the square root of the count of numbers in the list. Create this function and test it out using this data: http://www.ishelp.info/data/housing_full.csv

To check your work, input the column "SalePrice" into your function. If you created it correctly, the first 10 values returned (out of 39) should be as follows:

```
# Output:  
# [34900, 53745, 72590, 91435, 110280, 129125, 147970, 166815, 185660, 204505, ...]
```

This example is easy to follow. However, most data will not come pre-sorted and easy to understand, like the example above. Rather, it will come in whatever order the data was collected. A nicely sorted list is not particularly useful; what you need is a list of recoded values in the order that they were originally inputted. For example, the two lists above would more likely be inputted and outputted as follows:

```
original_list = [92, 3, 44, 39, 1, 36, 1, 68, 70]  
binned_list = [92, 31, 61, 61, 31, 61, 31, 92, 92]
```

Therefore, your next task is to extend your function to allow the caller to decide whether they want that list of unique bin edges you outputted previously or a full list of all recoded values where the n of the new list equals the n of the inputted list. To check your work, the first 10 records of

this technique based on the SalePrice variable in the housing_full.csv dataset should look like this:

```
# Output:  
# [223350, 185660, 242195, 147970, 261040, 147970, 317575, 204505, 147970, 129125, ...]
```

You are doing great! However, the square root rule is super old, and there are better techniques these days. In fact, including the original rule, there are many theories for determining the number of bins required:

- Square root (original date ???): oldie but goodie
 - (number of) Bins = square root of the count of values ($\sqrt{n}$)
 - Bin width = $(\max - \min) / \sqrt{n}$
- Sturges (1926)
 - Bins = $\log_2 n + 1$
 - Bin width = $(\max - \min) / \text{ceil}(\log_2 n) + 1$
- Rice (1944)
 - Bins = $2 * \text{cubed root of the sample size } (\sqrt[3]{n})$
 - Bin width = $(\max - \min) / (2 * \sqrt[3]{n})$
- Scott (1979)
 - Bins = $(\max - \min) / 3.5 * \text{std} / \sqrt[3]{n}$
 - Bin width = $3.5 * \text{std} / \sqrt[3]{n}$
- Freedman-Diaconis (1981)
 - Bins = $(\max - \min) / 2 * \text{IQR} / \sqrt[3]{n}$
 - Bin width = $2 * \text{IQR} / \sqrt[3]{n}$
 - Interquartile range (IQR) = quartile 3 (75% quantile) - quartile 1 (25% quantile) = $Q3 - Q1$
- Variable-width bins (2006)

- $\text{Bins} = 2 * n^{(2/5)}$
- `Bin width = np.quantile(list, 1 / bins)`

In other words, you need to mark variable bin edges by calculating 1/bins equal quantiles. So, if you need 5 bins, then calculate quantiles for 0.20, 0.40, 0.60, 0.80 and include the max (`quantile(1.0)`)

- Use this formula to eliminate skewness problems

Your task is to update your function yet again to allow the user to choose which binning theory they want to use or return a DataFrame of all five. You should also give them the option to just return the sorted, unique list of edges for all five theories. Here is a sample data output based on the first five records of the SalePrice variable in the `housing_full.csv` dataset so that you can check your work. The number of bins for each theory is included in parentheses in the column labels:

	sqrt (38)	sturges (11)	rice (22)	scott (29)	freedman-diaconis (48)	variable-width (36)
0	223350	222559	225322	230908	212656	28
1	185660	222559	193585	181906	183030	23
2	242195	285112	225322	230908	227469	29
3	147970	160006	161848	157405	153404	13
4	261040	285112	257059	255409	257095	32

Figure 6.2: Preview of Function Call with All Binning Theories Implemented on Full Dataset

By the way, if you completed this practice problem and created this function, you should add it to your tool kit. The first five binning methods above are common techniques you can use to generate histograms and bar charts, and the last technique is a good method to resolve skewness issues.



The next practice problem is more advanced. You may need to use AI to help you design, debug, and test your solution. If you do, make sure you

understand every line of code you submit.

Practice #3: Datetimes in `unistats()`

Update your **`unistats()`** function so it can detect and summarize date/time columns. Many real datasets store dates as text (e.g., "2024-09-17") rather than as true datetime values.

Your function should (1) correctly identify datetime columns, including columns that are currently stored as strings, and then (2) add datetime-friendly outputs for those columns.

For each datetime column, add these outputs to the `unistats` table:

- **MinDate** (earliest date)
- **MaxDate** (latest date)
- **DateRangeDays** (difference in days between max and min)
- **MostCommonYear** (the year that appears most often)

To test your work, create a small synthetic dataset that includes at least one numeric column, one categorical column, and one datetime column stored as strings. Then run **`unistats()`** and verify that the datetime column produces valid values in the new fields.



6.5Homework

Complete the assessment below:

This assessment can be taken [online](#).